Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Network

Lab Task 3

Last Date of Submission: 31th -Oct-2026

Submitted To: Sir Hassan Mujtaba

Student Name: Taj Uddin Khan

Registration Number: 24108234

***Instructions**: Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.*

CLO 2 – C 3

Screenshot of the code:

[28]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset

DATA_FILE = r"C:\Users\AK\Downloads\breast_cancer_wisconsin.csv"

df = pd.read_csv(DATA_FILE)

# Data Preprocessing

df = df.drop_duplicates()
df = df.dropna(subset=["diagnosis"])

df["diagnosis"] = df["diagnosis"].astype(str).str.strip().str.lower()

drop_columns = [
    col for col in df.columns
    if col.lower() in ["id", "unnamed: 32", "unnamed: 0"]
]

df = df.drop(columns=drop_columns, errors="ignore")

# Binary Label Encoding
# Benign = 0
# Malignant = 1

y = (df["diagnosis"] == "malignant").astype(int).values
```

```
X_df = df.drop(columns=["diagnosis"])

X_df = X_df.apply(pd.to_numeric, errors="coerce")
X_df = X_df.fillna(X_df.median())

X = X_df.values.astype(float)

# Train Test Split (80:20)

np.random.seed(42)

train_indices = []
test_indices = []

for label in np.unique(y):

    indices = np.where(y == label)[0]

    np.random.shuffle(indices)

    split = int(len(indices) * 0.8)

    train_indices.extend(indices[:split])
    test_indices.extend(indices[split:])

train_indices = np.array(train_indices)
test_indices = np.array(test_indices)

np.random.shuffle(train_indices)
np.random.shuffle(test_indices)

X_train = X[train_indices]
X_test = X[test_indices]

y_train = y[train_indices]
y_test = y[test_indices]
```

```
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

std[std == 0] = 1

X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

# Single Layer Perceptron From Scratch

class Perceptron:

    def __init__(self, learning_rate=0.001, epochs=3000):

        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0
        self.errors = []

    def activation(self, z):

        return (z >= 0).astype(int)

    def predict(self, X):

        z = np.dot(X, self.weights) + self.bias

        return self.activation(z)

    def train(self, X, y):

        samples, features = X.shape

        self.weights = np.zeros(features)
        self.bias = 0
```

```
        best_weights = self.weights.copy()
        best_bias = self.bias
        best_errors = samples + 1

        for epoch in range(self.epochs):

            indices = np.random.permutation(samples)

            errors = 0

            for i in indices:

                prediction = self.predict(X[i:i+1])[0]

                update = self.learning_rate * (
                    y[i] - prediction
                )

                if update != 0:

                    self.weights += update * X[i]
                    self.bias += update

                    errors += 1

            self.errors.append(errors)

            if errors < best_errors:

                best_errors = errors
                best_weights = self.weights.copy()
                best_bias = self.bias

            if errors == 0:

                break
```

```
        self.weights = best_weights
        self.bias = best_bias


# Create and Train Model

model = Perceptron(
    learning_rate=0.001,
    epochs=3000
)

model.train(X_train, y_train)

# Testing

y_pred = model.predict(X_test)

# Evaluation Metrics

TP = np.sum((y_test == 1) & (y_pred == 1))
TN = np.sum((y_test == 0) & (y_pred == 0))
FP = np.sum((y_test == 0) & (y_pred == 1))
FN = np.sum((y_test == 1) & (y_pred == 0))

accuracy = (TP + TN) / len(y_test)

precision = TP / (TP + FP) if TP + FP > 0 else 0

recall = TP / (TP + FN) if TP + FN > 0 else 0

f1_score = (
    2 * precision * recall / (precision + recall)
    if precision + recall > 0 else 0
)
```

```
# Display Evaluation Results

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall:", round(recall * 100, 2), "%")
print("F1-Score:", round(f1_score * 100, 2), "%")

# Confusion Matrix

cm = np.array([
    [TN, FP],
    [FN, TP]
])

print("\nConfusion Matrix:")
print(cm)

# Save Predictions

results = pd.DataFrame({

    "Actual": y_test,

    "Predicted": y_pred,

    "Actual Diagnosis": [
        "Malignant" if value == 1 else "Benign"
        for value in y_test
    ],

    "Predicted Diagnosis": [
        "Malignant" if value == 1 else "Benign"
        for value in y_pred
    ]

})
```

```
results.to_csv("test_predictions.csv", index=False)

# Training Error Graph

plt.figure(figsize=(8, 5))

plt.plot(model.errors)

plt.title("Perceptron Training Errors")
plt.xlabel("Epochs")
plt.ylabel("Number of Errors")
plt.grid(True)

plt.tight_layout()
plt.savefig("training_errors.png")
plt.show()

# Confusion Matrix Graph

plt.figure(figsize=(6, 5))

plt.imshow(cm, cmap="Blues")

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.xticks([0, 1], ["Benign", "Malignant"])
plt.yticks([0, 1], ["Benign", "Malignant"])
```

```
for i in range(2):

    for j in range(2):

        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center",
            fontsize=14
        )

plt.colorbar()
plt.tight_layout()

plt.savefig("confusion_matrix.png")
plt.show()
```

Output of the Code:

```
Accuracy: 93.04 %
Precision: 92.68 %
Recall: 88.37 %
F1-Score: 90.48 %

Confusion Matrix:
[[69  3]
 [ 5 38]]
```



